Chương 6

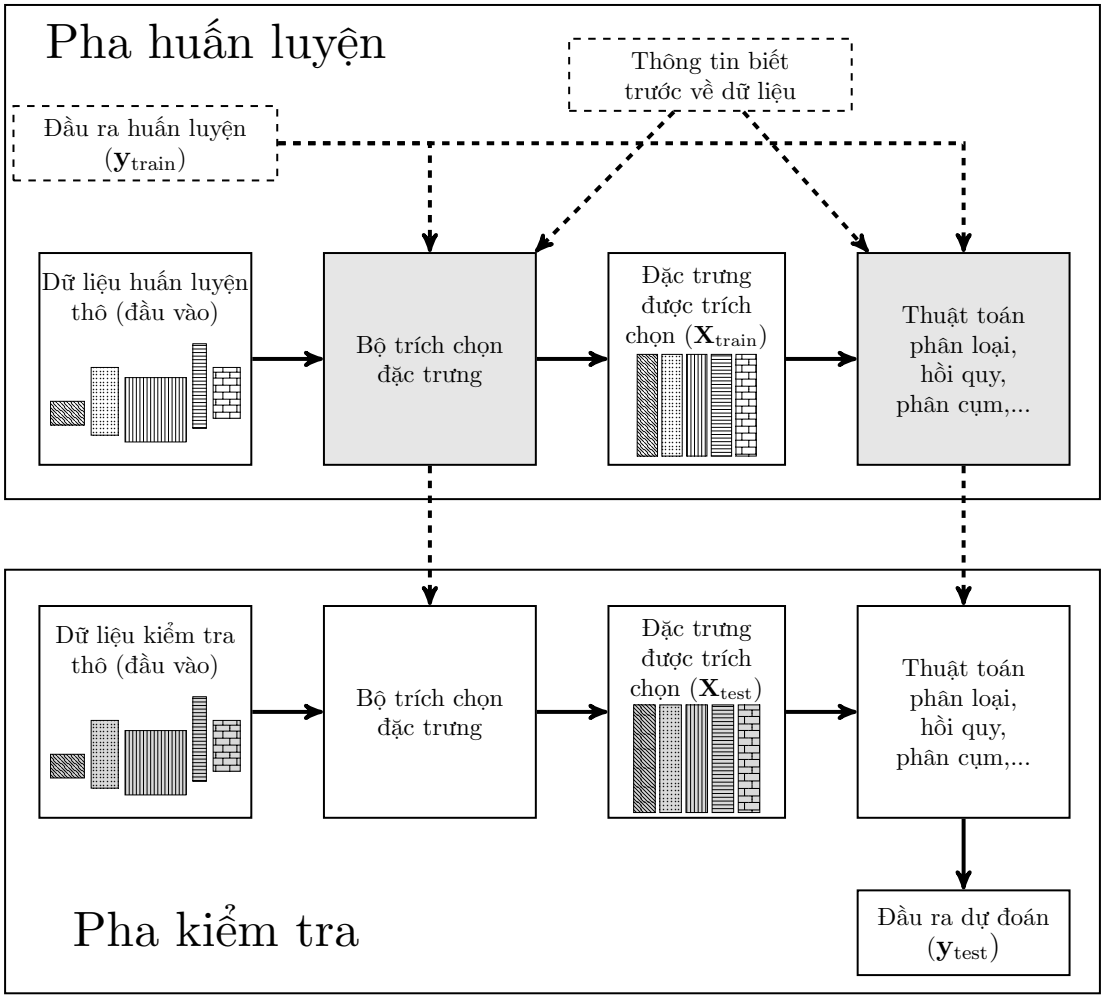
Các kỹ thuật xây dựng đặc trưng

6.1. Giới thiệu

Mỗi điểm dữ liệu trong một mô hình machine learning thường được biểu diễn bằng một vector được gọi là *vector đặc trưng* (feature vector). Trong cùng một mô hình, các vector đặc trưng của các điểm thường có kích thước như nhau. Điều này là cần thiết vì các mô hình bao gồm các phép toán với ma trận và vector, các phép toán này yêu cầu dữ liệu có chiều phù hợp. Tuy nhiên, dữ liệu thực tế thường ở dạng thô với kích thước khác nhau hoặc kích thước như nhau nhưng số chiều quá lớn gây trở ngại trong việc lưu trữ. Vì vậy, việc lựa chọn, tính toán đặc trưng phù hợp cho mỗi bài toán là một bước quan trọng.

Trong những bài toán thị giác máy tính, các bức ảnh thường là các ma trận hoặc mảng nhiều chiều với kích thước khác nhau. Các bức ảnh này có thể được chụp bởi nhiều camera trong các điều kiện ánh sáng khác nhau. Các bức ảnh này không những cần được đưa về kích thước phù hợp mà còn cần được chuẩn hoá để tăng hiệu quả của mô hình.

Trong các bài toán xử lý ngôn ngữ tự nhiên, độ dài văn bản có thể khác nhau, được viết theo những văn phong khác nhau. Trong nhiều trường hợp, việc thêm bớt một vài từ vào một văn bản có thể thay đổi hoàn toàn nội dung của nó. Hoặc cùng là một câu nói nhưng tốc độ, âm giọng của mỗi người là khác nhau, tại các thời điểm khác nhau là khác nhau. Khi làm việc với các bài toán machine learning, nhìn chung ta chỉ có được dữ liệu thô chưa qua chỉnh sửa và chọn lọc. Ngoài ra, ta có thể phải loại bỏ những dữ liệu nhiễu và đưa dữ liệu thô với kích thước khác nhau về cùng một chuẩn. Dữ liệu chuẩn này phải đảm bảo giữ được những thông tin đặc trưng của dữ liệu thô ban đầu. Không những thế, ta cần thiết kế những phép biến đổi để có những đặc trưng phù hợp cho từng bài toán.



Hình 6.1. Mô hình chung trong các bài toán machine learning

Quá trình quan trọng này được gọi là *trích chọn đặc trưng* (feature extraction hoặc feature engineering).

Để có cái nhìn tổng quan, chúng ta cần đặt bước trích chọn đặc trưng này trong cả quy trình xây dựng một mô hình machine learning.

6.2. Mô hình chung cho các bài toán machine learning

Phần lớn các mô hình machine learning có thể được minh hoạ trong Hình 6.1. Có hai pha lớn trong mỗi bài toán machine learning là *pha huấn luyện* (training phase) và *pha kiểm tra* (test phase). Pha huấn luyện xây dựng mô hình dựa trên dữ liệu huấn luyện. Dữ liệu kiểm tra được sử dụng để đánh giá hiệu quả mô hình[14].

[14] Trước khi đánh giá một mô hình trên tập kiểm tra, ta cần đảm bảo rằng mô hình đó đã làm việc tốt trên tập huấn luyện.

6.2.1. Pha huấn luyện

Có hai khối có nền màu xám cần được thiết kế:

Khối trích chọn đặc trưng có nhiệm vụ tạo ra một vector đặc trưng cho mỗi điểm dữ liệu đầu vào. Vector đặc trưng này thường có kích thước như nhau, bất kể dữ liệu đầu vào có kích thước như thế nào.

Đầu vào của khối trích chọn đặc trưng có thể là các yếu tố sau:

- *Dữ liệu huấn luyện đầu vào ở dạng thô* bao gồm tất cả các thông tin ban đầu. Ví dụ, dữ liệu thô của một ảnh là giá trị của từng điểm ảnh, của một văn bản là từng từ, từng câu; của một file âm thanh là một đoạn tín hiệu; của thời tiết là thông tin về hướng gió, nhiệt độ, độ ẩm không khí,... Dữ liệu thô này thường không ở dạng vector, không có số chiều như nhau hoặc một vài thông tin bị khuyết. Thậm chí chúng có thể có số chiều như nhau nhưng rất lớn. Chẳng hạn, một bức ảnh màu kích thước 1000×1000 có số điểm ảnh là 3×10^{6}[15]. Đây là một con số quá lớn, không có lợi cho lưu trữ và tính toán.

- *Dữ liệu huấn luyện đầu ra*: dữ liệu này có thể được sử dụng hoặc không. Trong các thuật toán học không giám sát, ta không biết đầu ra nên hiển nhiên không có giá trị này. Trong các thuật toán học có giám sát, đôi khi dữ liệu này cũng không được sử dụng. Ví dụ, việc giảm chiều dữ liệu có thể không cần sử dụng dữ liệu đầu ra. Nếu dữ liệu đầu vào đã là các vector cột cùng chiều, ta chỉ cần nhân vào bên phải của chúng một ma trận chiếu ngẫu nhiên. Ma trận này có số hàng ít hơn số cột để đảm bảo số chiều thu được nhỏ hơn số chiều ban đầu. Việc làm này mặc dù làm mất đi thông tin, trong nhiều trường hợp vẫn mang lại hiệu quả vì đã giảm được lượng tính toán ở phần sau. Đôi khi ma trận chiếu không phải là ngẫu nhiên mà có thể được *học* dựa trên toàn bộ dữ liệu thô ban đầu (xem Chương 21).

  Trong nhiều trường hợp khác, dữ liệu đầu ra của tập huấn luyện cũng được sử dụng để tạo bộ trích chọn đặc trưng. Việc giữ lại nhiều thông tin không quan trọng bằng việc giữ lại các thông tin có ích. Ví dụ, dữ liệu thô là các hình vuông và hình tam giác màu đỏ và xanh. Trong bài toán phân loại đa giác, nếu các nhãn là *tam giác* và *vuông*, ta không quan tâm tới màu sắc mà chỉ quan tâm tới số cạnh của đa giác. Ngược lại, trong bài toán phân loại màu với các nhãn là *xanh* và *đỏ*, ta không quan tâm tới số cạnh mà chỉ quan tâm đến màu sắc.

- *Các thông tin biết trước về dữ liệu*: Ngoài dữ liệu huấn luyện, các thông tin biết trước ngoài lề cũng có tác dụng trong việc xây dựng bộ trích chọn đặc trưng. Chẳng hạn, có thể dùng các bộ lọc để giảm nhiễu nếu dữ liệu là âm

[15] Ảnh màu thường có ba kênh: red, green, blue – RGB

thanh, hoặc dùng các bộ dò cạnh để tìm ra cạnh của các vật thể trong dữ liệu ảnh. Nếu dữ liệu là ảnh các tế bào và ta cần đưa ảnh về kích thước nhỏ hơn, ta cần lưu ý về độ phân giải của tế bảo của ảnh trong kích thước mới. Ta cần xây dựng một bộ trích chọn đặc trưng phù hợp với từng loại dữ liệu.

Sau khi xây dựng bộ trích chọn đặc trưng, dữ liệu thô ban đầu được đưa qua và tạo ra các vector đặc trưng tương ứng gọi là *đặc trưng đã trích xuất* (extracted feature). Những đặc trưng này được dùng để huấn luyện các thuật toán machine learning chính như phân loại, phân cụm, hồi quy,... trong khối màu xám thứ hai.

Trong một số thuật toán cao cấp hơn, việc xây dựng bộ trích chọn đặc trưng và các thuật toán chính có thể được thực hiện đồng thời thay vì riêng lẻ như trên. Đầu vào của toàn bộ mô hình là dữ liệu thô hoặc dữ liệu thô đã qua một bước xử lý nhỏ. Các mô hình đó có tên gọi chung là 'mô hình đầu cuối' (end-to-end model). Với sự phát triển của deep learning trong những năm gần đây, người ta cho rằng các mô hình đầu cuối mang lại kết quả tốt hơn nhờ vào việc hai khối được huấn luyện cùng nhau, bổ trợ lẫn nhau cùng hướng tới mục đích chung cuối cùng. Thực tế cho thấy, các mô hình machine learning hiệu quả nhất thường là các mô hình đầu cuối.

6.2.2. Pha kiểm tra

Ở pha kiểm tra, vector đặc trưng của một điểm dữ liệu thô mới được tạo bởi bộ trích chọn đặc trưng thu được từ pha huấn luyện. Vector đặc trưng này được đưa vào thuật toán chính đã tìm được để đưa ra quyết tra. Có một lưu ý quan trọng là khi xây dựng bộ trích chọn đặc trưng và các thuật toán chính, ta không được sử dụng dữ liệu kiểm tra. Các công việc đó được thực hiện chỉ dựa trên dữ liệu huấn luyện.

6.3. Một số kỹ thuật trích chọn đặc trưng

6.3.1. Trực tiếp lấy dữ liệu thô

Xét bài toán với dữ liệu là các bức ảnh xám có kích thước cố tra $m \times n$ điểm ảnh. Cách đơn giản nhất để tạo ra vector đặc trưng cho bức ảnh này là xếp chồng các cột của ma trận điểm ảnh để được một vector $m \times n$ phần tử. Vector này có thể được coi là vector đặc trưng với mỗi đặc trưng là giá trị của một điểm ảnh. Việc làm đơn giản này đã làm mất thông tin về vị trí tương đối giữa các điểm ảnh vì các điểm ảnh gần nhau theo phương ngang trong bức ảnh ban đầu không còn gần nhau trong vector đặc trưng. Tuy nhiên, trong nhiều trường hợp, kỹ thuật này vẫn mang lại kết quả khả quan.

6.3.2. Lựa chọn đặc trưng

Đôi khi, việc trích chọn đặc trưng đơn giản là chọn ra các thành phần phù hợp trong dữ liệu ban đầu. Việc làm này thường xuyên được áp dụng khi một lượng dữ liệu thu được không có đầy đủ các thành phần hoặc dữ liệu có quá nhiều chiều mà phần lớn không mang nhiều thông tin hữu ích.

6.3.3. Giảm chiều dữ liệu

Giả sử dữ liệu ban đầu là một vector $\mathbf{x} \in \mathbb{R}^D$, $\mathbf{A}$ là một ma trận trong $R^{d \times D}$ và $\mathbf{z} = \mathbf{A}\mathbf{x} \in \mathbb{R}^d$. Nếu $d < D$, ta thu được một vector với số chiều nhỏ hơn. Đây là một kỹ thuật phổ biến trong giảm chiều dữ liệu. Ma trận $\mathbf{A}$ được gọi là *ma trận chiếu* (projection matrix), có thể là một ma trận ngẫu nhiên. Tuy nhiên, việc chọn một ma trận chiếu ngẫu nhiên đôi khi mang lại kết quả tệ không mong muốn vì thông tin có thể bị thất thoát quá nhiều. Một phương pháp phổ biến để tối thiểu lượng thông tin mất đi có tên là *phân tích thành phần chính* (principal component analysis) sẽ được trình bày trong Chương 21.

Lưu ý: Kỹ thuật xây dựng đặc trưng không nhất thiết luôn làm giảm số chiều dữ liệu, đôi khi vector đặc trưng có thể có có kích thước lớn hơn dữ liệu thô ban đầu nếu việc này mang lại hiệu quả tốt hơn.

6.3.4. Túi từ

Chúng ta hẳn đã tự đặt ra các câu hỏi: với một văn bản, vector đặc trưng sẽ có dạng như thế nào? Làm sao đưa các từ, các câu, đoạn văn ở dạng ký tự trong các văn bản về một vector mà mỗi phần tử là một số?

Có một kỹ thuật rất phổ biến trong xử lý văn bản có tên là *túi từ* (bag of words, BoW).

Bắt đầu bằng ví dụ phân loại tin nhắn rác. Nhận thấy rằng nếu một tin có chứa các từ "khuyến mại", "giảm giá", "trúng thưởng", "miễn phí", "quà tặng", "tri ân",..., nhiều khả năng đó là một tin nhắn rác. Từ đó, phương pháp đầu tiên có thể nghĩ tới là đếm số lần các từ này xuất hiện, nếu số lượng này nhiều hơn một ngưỡng nào đó thì ta quyết định đó là tin rác[16]. Với các loại văn bản khác nhau, lượng từ liên quan tới từng chủ đề cũng khác nhau. Từ đó có thể dựa vào số lượng các từ trong từng loại để tạo các vector đặc trưng cho từng văn bản.

Xin lấy một ví dụ về hai văn bản đơn giản sau đây[17]:

```
(1) "John likes to watch movies. Mary likes movies too."
```

[16] Bài toán thực tế phức tạp hơn khi các tin nhắn có thể được viết dưới dạng không dấu, bị cố tình viết sai chính tả, hoặc dùng các ký tự đặc biệt

[17] *Bag of words – Wikipedia* (https://goo.gl/rBtZqx)

và

```
(2) "John also likes to watch football games."
```

Dựa trên hai văn bản này, ta có danh sách các từ được sử dụng, được gọi là *từ điển* (dictionary hoặc codebook) với mười *từ* như sau:

```
["John", "likes", "to", "watch", "movies", "also", "football", "games", "
    Mary", "too"]
```

Với mỗi văn bản, ta sẽ tạo ra một vector đặc trưng có số chiều bằng 10, mỗi phần tử đại diện cho số từ tương ứng xuất hiện trong văn bản đó. Với hai văn bản trên, ta sẽ có hai vector đặc trưng:

```
(1) [1, 2, 1, 1, 2, 0, 0, 0, 1, 1]
(2) [1, 1, 1, 1, 0, 1, 1, 1, 0, 0]
```

Văn bản (1) có một từ `"John"`, hai từ `"likes"`, không từ `"also"`, không từ `"football"`,... nên ta thu được vector tương ứng như trên.

Có một vài điều cần lưu ý trong BoW:

- Với những ứng dụng thực tế, từ điển có số lượng từ lớn hơn rất nhiều, có thể đến cả triệu, như vậy vector đặc trưng thu được sẽ rất dài. Một văn bản chỉ có một câu, và một tiểu thuyết nghìn trang đều được biểu diễn bằng các vector có kích thước như nhau.

- Có rất nhiều từ trong từ điển không xuất hiện trong một văn bản. Như vậy các vector đặc trưng thu được thường có nhiều phần tử bằng không. Các vector đó được gọi là *vector thưa* (sparse vector). Để việc lưu trữ được hiệu quả hơn, ta không lưu mọi thành phần của một vector thưa mà chỉ lưu vị trí của các phần tử khác không và giá trị tương ứng. Chú ý rằng nếu có hơn một nửa số phần tử khác không, việc làm này lại phản tác dụng. Tuy nhiên, trường hợp này ít xảy ra vì hiếm có văn bản chứa tới một nửa số từ trong từ điển.

- Các từ hiếm gặp được xử lý như thế nào? Một kỹ thuật thường dùng là thêm phần tử `<Unknown>` vào trong từ điển. Mọi từ không có trong từ điển đều được coi là `<Unknown>`.

- Tuy nhiên, những từ hiếm đôi khi lại mang những thông tin quan trọng nhất mà chỉ loại văn bản đó có. Đây là một nhược điểm của BoW. Có một phương pháp cải tiến giúp khắc phục nhược điểm này tên là *term frequency-inverse*

document frequency (TF-IDF) [SWY75] dùng để xác định tầm quan trọng của một từ trong một văn bản dựa trên toàn bộ văn bản trong cơ sở dữ liệu[18].

- Nhược điểm lớn nhất của BoW là nó không mang thông tin về thứ tự của các từ, cũng như sự liên kết giữa các câu, các đoạn văn trong văn bản. Thứ tự của các từ trong văn bản thường mang thông tin quan trọng. Ví dụ, ba câu sau đây: "Em yêu anh không?", "Em không yêu anh", và "Không, (nhưng) anh yêu em" khi được trích chọn đặc trưng bằng BoW sẽ cho ra ba vector giống hệt nhau, mặc dù ý nghĩa khác hẳn nhau.

6.3.5. BoW cho dữ liệu ảnh

BoW cũng được áp dụng cho các bức ảnh với cách định nghĩa *từ* và *từ điển* khác. Xét các ví dụ sau:

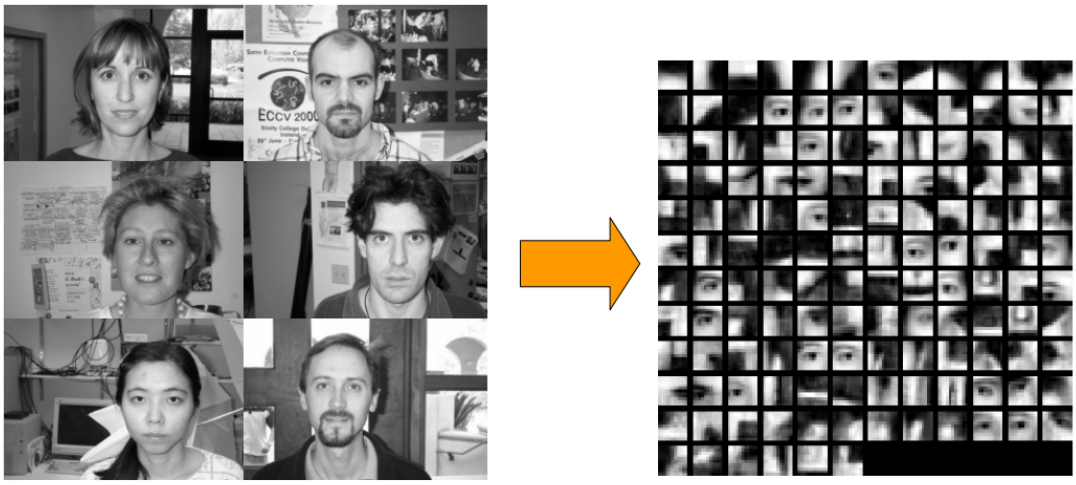
Ví dụ 1: Giả sử có một tập dữ liệu ảnh có hai nhãn là rừng và sa mạc, và một bức ảnh chỉ rơi vào một trong hai loại này. Việc phân loại một bức ảnh là rừng hay sa mạc một cách tự nhiên nhất là dựa vào màu sắc. Màu xanh lục nhiều tương ứng với rừng, màu đỏ và vàng nhiều tương ứng với sa mạc. Ta có một mô hình đơn giản để trích chọn đặc trưng như sau:

- Với một bức ảnh, chuẩn bị một vector $\mathbf{x}$ có số chiều bằng 3, đại diện cho ba màu xanh lục (x_1), đỏ (x_2), và vàng (x_3).
- Với mỗi điểm ảnh trong bức ảnh đó, xem nó gần với màu xanh, đỏ hay vàng nhất dựa trên giá trị của điểm ảnh đó. Nếu nó gần điểm xanh nhất, tăng x_1 lên một; gần đỏ nhất, tăng x_2 lên một; gần vàng nhất, tăng x_3 lên một.
- Sau khi xem xét tất cả các điểm ảnh, dù cho bức ảnh có kích thước thế nào, ta vẫn thu được một vector có kích thước bằng ba, mỗi phần tử thể hiện việc có bao nhiêu điểm ảnh trong bức ảnh có màu tương ứng. Vector cuối này còn được gọi là *histogram vector* của bức ảnh và có thể coi là một vector đặc trưng tốt trong bài toán phân loại ảnh rừng hay say mạc.

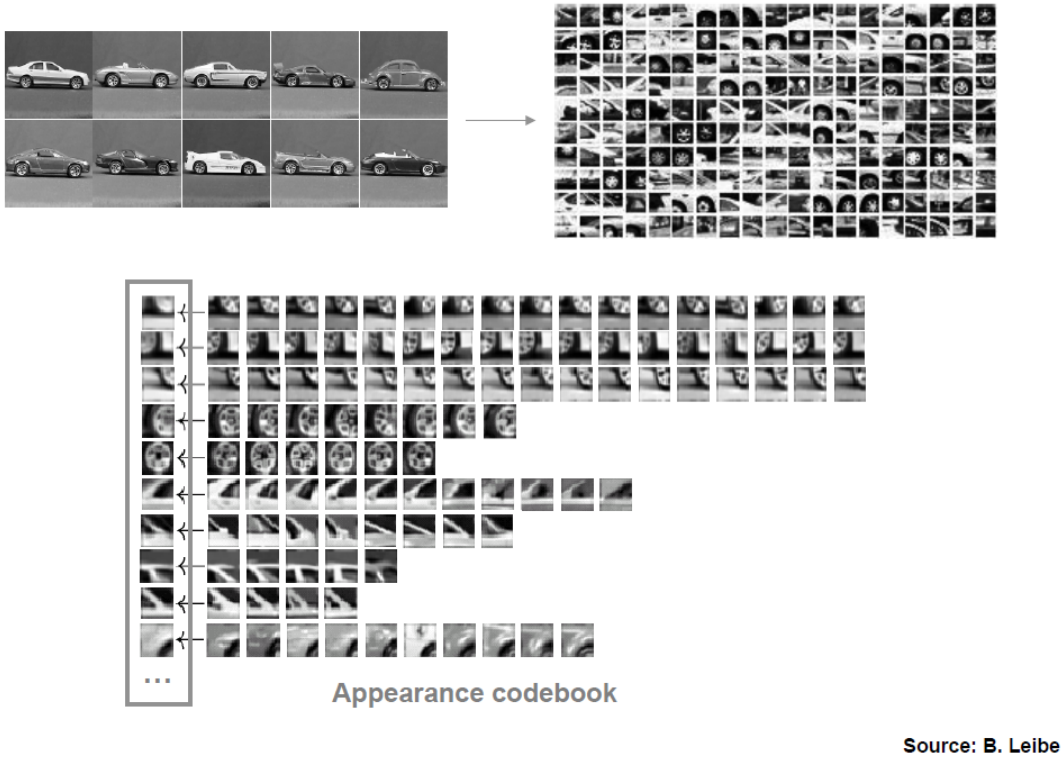
Ví dụ 2: Trên thực tế, các bài toán xử lý ảnh không đơn giản như trong ví dụ trên đây. Mắt người thực ra nhạy với các đường nét, hình dáng hơn là màu sắc. Chúng ta có thể nhận biết được một bức ảnh có cây hay không ngay cả khi bức ảnh đó không có màu. Vì vậy, xem xét giá trị từng điểm ảnh không mang lại kết quả khả quan vì lượng thông tin về đường nét đã bị mất.

Có một giải pháp là thay vì xem xét một điểm ảnh, ta xem xét một vùng hình chữ nhật nhỏ trong ảnh, vùng này còn được gọi là *patch*. Các patch này nên đủ lớn để có thể chứa được các bộ phận đặc tả vật thể trong ảnh. Ví dụ với mặt

[18] *5 Algorithms Every Web Developer Can Use and Understand, section 5* (https://goo.gl/LJW3H1).



Hình 6.2. Bag of words cho ảnh chứa mặt người (Nguồn: *Bag of visual words model: recognizing object categories* (https://goo.gl/EN2oSM).



Hình 6.3. Bag of Words cho ảnh xe hơi (Nguồn: B. Leibe).

người, các patch cần chứa được các phần của khuôn mặt như mắt, mũi, miệng (xem Hình 6.2). Tương tự, với ảnh là ô tô, các patch thu được có thể là bánh xe, khung xe, cửa xe,...(xem Hình 6.3, hàng trên bên phải).

Trong xử lý văn bản, hai từ được coi là như nhau nếu nó được biểu diễn bởi các ký tự giống nhau. Câu hỏi đặt ra là, trong xử lý ảnh, hai patch được coi là như nhau khi nào? Khi mọi điểm ảnh trong hai patch có giá trị bằng nhau sao?

Câu trả lời là không. Xác suất để hai patch giống hệt nhau từng điểm ảnh là rất thấp vì có thể một phần của vật thể trong một patch bị lệch đi, bị méo, hoặc có độ sáng thay đổi. Trong những trường hợp này, mặc dù mắt người vẫn thấy hai patch đó rất giống nhau, máy tính có thể nghĩ đó là hai patch khác nhau. Vậy, hai patch được coi là như nhau khi nào? Từ và từ điển ở đây được định nghĩa như thế nào?

Ta có thể áp dụng một phương pháp phân cụm đơn giản là K-means (xem Chương 10) để tạo ra từ điển và coi hai patch là gần nhau nếu khoảng cách Euclid giữa hai vector tạo bởi hai patch là nhỏ. Với rất nhiều patch thu được, giả sử cần xây dựng một từ điển với chỉ khoảng 1000 *từ*, ta có thể dùng phân cụm K-means để phân toàn bộ các patch thành 1000 cụm (mỗi cụm được coi là một *bag*) khác nhau. Mỗi cụm gồm các patch gần giống nhau, được mô tả bởi trung bình cộng của tất cả các patch trong cụm đó (xem Hình 6.3 hàng dưới). Với một ảnh bất kỳ, ta trích ra các patch từ ảnh đó, tìm xem mỗi patch gần với cụm nào nhất trong 1000 cụm tìm được ở trên và quyết định patch này thuộc cụm đó. Cuối cùng, ta sẽ thu được một vector đặc trưng có kích thước bằng 1000 mà mỗi phần tử là số lượng các patch trong ảnh rơi vào cụm tương ứng.

6.4. Học chuyển tiếp cho bài toán phân loại ảnh

Mục này được viết trên cơ sở bạn đọc đã có kiến thức nhất định và deep learning.

Ngoài BoW, các phương pháp phổ biến được sử dụng để xây dựng vector đặc trưng cho ảnh là *scale invariant feature transform – SIFT* [Low99], *speeded-up robust features – SURF* [BTVG06], *histogram of oriented gradients – HOG* [DT05], *local binary pattern – LBP* [Low99],... Các bộ phân loại thường được sử dụng là SVM đa lớp (Chương 29), hồi quy softmax (Chương 15), mã hóa thưa và học từ điển [WYG+09, VMM+16, VM17], rừng ngẫu nhiên [LW+02],...

Các đặc trưng được tạo bởi các phương pháp nêu trên thường được gọi là các *đặc trưng thủ công* (hand-crafted feature) vì chúng chủ yếu dựa trên các quan sát về đặc tính riêng của ảnh và được xây dựng chung cho mọi loại dữ liệu ảnh. Các phương pháp này cho kết quả khá ấn tượng trong một số trường hợp. Tuy nhiên, chúng vẫn còn nhiều hạn chế vì quá trình tìm ra các đặc trưng và các bộ phân loại là riêng biệt. Hơn nữa, các bộ trích chọn này chỉ tìm ra các *đặc trưng mức thấp* (low-level features) của ảnh.

Những năm gần đây, deep learning phát triển cực nhanh dựa trên lượng dữ liệu huấn luyện khổng lồ và khả năng tính toán ngày càng được cải tiến của các máy tính. Kết quả cho bài toán phân loại ảnh ngày càng được nâng cao. Bộ

cơ sở dữ liệu thường được dùng nhất là ImageNet (https://www.image-net.org) với 1.2 triệu ảnh cho 1000 nhãn khác nhau. Rất nhiều mô hình deep learning đã giành chiến thắng trong các cuộc thi *ImageNet large scale visual recognition challenge – ILSVRC* (https://goo.gl/1A8drd): AlexNet [KSH12], ZFNet [ZF14], GoogLeNet [SLJ+15], ResNet [HZRS16], VGG [SZ14]. Nhìn chung, các mô hình này là các *mạng neuron đa tầng* (multi-layer neural network). Các tầng phía trước thường là các *tầng tích chập* (convolutional layer). Tầng cuối cùng là một *tầng nối kín* (fully connected layer) và thường là một bộ hồi quy softmax (xem Hình 6.4). Vì vậy đầu ra của tầng gần cuối cùng có thể được coi là vector đặc trưng và hồi quy softmax chính là bộ phân loại được sử dụng[19].

Việc bộ trích chọn đặc trưng và bộ phân loại được huấn luyện cùng nhau thông qua tối ưu hệ số trong mạng neuron sâu khiến các mô hình này đạt kết quả tốt. Tuy nhiên, những mô hình này đều bao gồm rất nhiều tầng các trọng số. Việc huấn luyện dựa trên hơn một triệu bức ảnh tốn rất nhiều thời gian (2-3 tuần).

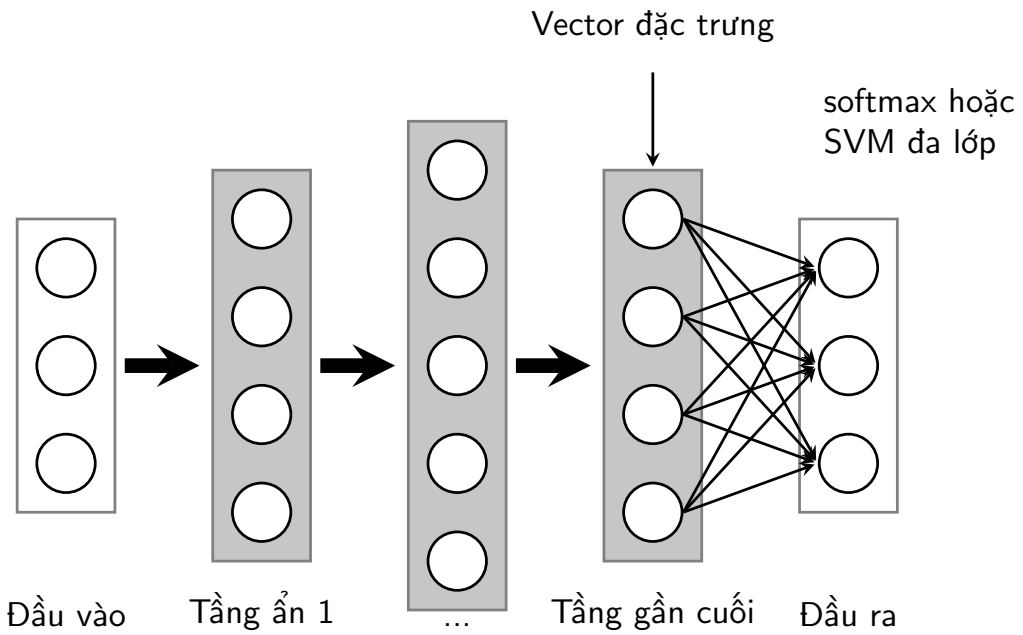
Với các bài toán phân loại các dữ liệu ảnh khác với tập huấn luyện nhỏ, ta có thể không cần xây dựng lại mạng neuron và huấn luyện nó từ đầu. Thay vào đó, ta có thể sử dụng các mô hình đã được huấn luyện nêu trên và thay đổi kiến trúc của mạng cho phù hợp. Phương pháp sử dụng các mô hình có sẵn như vậy còn được gọi là *học chuyển tiếp* (transfer learning).

Toàn bộ các tầng trừ tầng đầu ra có thể được coi là một bộ trích chọn đặc trưng. Điều này được rút ra dựa trên nhận xét rằng các bức ảnh thường có những đặc tính giống nhau. Sau đó, ta huấn luyện một bộ phân loại khác dựa trên vector đặc trưng đã đã được trích chọn. Cách làm này có thể tăng độ chính xác phân loại lên đáng kể so với việc sử dụng các đặc trưng thủ công vì các mạng neuron sâu được cho là có khả năng trích chọn các *đặc trưng mức cao* (high-level features) của ảnh.

Hướng tiếp cận thứ hai là sử dụng các mô hình đã được huấn luyện và cho huấn luyện thêm một vài tầng cuối dựa trên dữ liệu mới. Kỹ thuật này được gọi là *tinh chỉnh* (fine-tuning). Việc này được thực hiện dựa trên quan sát rằng những tầng đầu trong mạng neuron sâu trích xuất những đặc trưng chung mức thấp của đa số ảnh, các tầng cuối giúp trích chọn các đặc trưng mức cao phù hợp cho từng cơ sở dữ liệu (CSDL). Các đặc trưng mức cao có thể khác nhau tuỳ theo từng CSDL. Vì vậy, khi có dữ liệu mới, ta chỉ cần huấn luyện mạng neuron để trích chọn các đặc trưng mức cao phù hợp với dữ liệu mới này.

Dựa trên kích thước và sự giống nhau giữa CSDL mới và CSDL gốc (dùng để huấn luyện mạng neuron ban đầu), có một vài quy tắc để huấn luyện mạng neuron mới[20]:

[19] hồi quy softmax là một thuật toán phân loại, tên gọi *hồi quy* của nó mang tính lịch sử.
[20] *Transfer Learning, CS231n* (https://goo.gl/VN1g7F)



Hình 6.4. Kiến trúc deep learning cơ bản cho bài toán phân loại. Tầng cuối cùng là một tầng nối kín và thường là một hồi quy softmax.

- *CSDL mới nhỏ, tương tự CSDL gốc.* Vì CSDL mới nhỏ, việc tiếp tục huấn luyện mô hình có thể dễ dẫn đến hiện tượng *quá khớp* (overfitting, xem Chương 8). Cũng vì hai CSDL tương tự nhau, ta dự đoán rằng các đặc trưng mức cao của chúng tương tự nhau. Vì vậy, ta không cần huấn luyện lại mạng neuron mà chỉ cần huấn luyện một bộ phân loại dựa trên các vector đặc trưng thu được.

- *CSDL mới lớn, tương tự CSDL gốc.* Vì CSDL này lớn, quá khớp ít xảy ra hơn, ta có thể huấn luyện mô hình thêm một vài vòng lặp. Việc huấn luyện có thể được thực hiện trên toàn bộ hoặc chỉ một vài tầng cuối.

- *CSDL mới nhỏ, rất khác CSDL gốc.* Vì CSDL này nhỏ, tốt hơn hết là dùng các bộ phân loại đơn giản khác để tránh quá khớp. Nếu muốn sử dụng mạng neuron cũ, ta cũng chỉ nên tinh chỉnh các tầng cuối của nó. Hoặc có thể coi đầu ra của một tầng ở giữa của mạng neuron là vector đặc trưng rồi huấn luyện thêm một bộ phân loại.

- *CSDL mới lớn rất khác CSDL gốc.* Thực tế cho thấy, sử dụng các mạng neuron sẵn có trên CSDL mới vẫn hữu ích. Trong trường hợp này, ta vẫn có thể sử dụng các mạng neuron sẵn có như là điểm khởi tạo của mạng neuron mới, không nên huấn luyện mạng neuron mới từ đầu.

Một điểm đáng chú ý là khi tiếp tục huấn luyện các mạng neuron này, ta chỉ nên chọn tốc độ học nhỏ để các hệ số mới không đi quá xa so với các hệ số đã được huấn luyện ở các mô hình trước.

6.5. Chuẩn hoá vector đặc trưng

Các điểm dữ liệu đôi khi được đo đạc bằng những đơn vị khác nhau, chẳng hạn mét và feet. Đôi khi, hai thành phần của dữ liệu ban đầu chênh lệch nhau lớn, chẳng hạn một thành phần có khoảng giá trị từ 0 đến 1000, thành phần kia chỉ có khoảng giá trị từ 0 đến 1. Lúc này, chúng ta cần chuẩn hóa dữ liệu trước khi thực hiện các bước tiếp theo.

Chú ý : việc chuẩn hóa này chỉ được thực hiện khi vector dữ liệu đã có cùng chiều.

Sau đây là một vài phương pháp chuẩn hóa thường dùng.

6.5.1. Chuyển khoảng giá trị

Phương pháp đơn giản nhất là đưa tất cả các đặc trưng về cùng một khoảng, ví dụ $[0, 1]$ hoặc $[-1, 1]$. Để muốn đưa đặc trưng thứ i của một vector đặc trưng $\mathbf{x}$ về khoảng $[0, 1]$, ta sử dụng công thức

$$x_i' = \frac{x_i - \min(x_i)}{\max(x_i) - \min(x_i)}$$

trong đó x_i và x_i' lần lượt là giá trị đặc trưng ban đầu và giá trị đặc trưng sau khi được chuẩn hóa. $\min(x_i), \max(x_i)$ là giá trị nhỏ nhất và lớn nhất của đặc trưng thứ i xét trên toàn bộ dữ liệu huấn luyện.

6.5.2. Chuẩn hoá theo phân phối chuẩn

Một phương pháp khác thường được sử dụng là đưa mỗi đặc trưng về dạng một phân phối chuẩn có kỳ vọng là 0 và phương sai là 1. Công thức chuẩn hóa là

$$x_i' = \frac{x_i - \bar{x}_i}{\sigma_i}$$

với $\bar{x}_i, \sigma_i$ lần lượt là kỳ vọng và độ lệch chuẩn của đặc trưng đó xét trên toàn bộ dữ liệu huấn luyện.

6.5.3. Chuẩn hoá về cùng norm

Một lựa chọn khác cũng được sử dụng rộng rãi là biến vector dữ liệu thành vector có độ dài Euclid bằng một. Việc này có thể được thực hiện bằng cách chia mỗi vector đặc trưng cho ℓ_2 norm của nó:

$$\mathbf{x}' = \frac{\mathbf{x}}{\|\mathbf{x}\|_2}$$